Applied Information Security

The RSA Cryptosystem: Foundations, Attacks, and Implementation

Dr. Faisal Aslam

Contents

1 Lesson 1: The RSA Cryptosystem I — Foundations, Correctness, and Factoring **3**

1.1 Refresher: The Two Theorems RSA Is Built On . . . 4
1.1.1 Euler's Totient Function and Euler's Theorem . . . 4
1.1.2 The Chinese Remainder Theorem . . . 5
1.2 RSA Construction . . . 6
1.2.1 Key Generation . . . 6
1.2.2 Fast Modular Exponentiation . . . 6
1.2.3 CRT-Based Decryption Speedup . . . 7
1.3 Correctness of RSA . . . 8
1.4 Security Foundations . . . 9
1.4.1 The RSA Problem and the RSA Assumption . . . 9
1.4.2 Why Textbook RSA Is Not Secure As-Is . . . 9
1.5 Practical Key Generation . . . 11
1.5.1 Primality Testing . . . 11
1.5.2 Choosing e and d . . . 11
1.5.3 Prime Generation Pitfalls . . . 11
1.6 Factoring Algorithms . . . 13
1.6.1 Trial Division and Fermat's Method . . . 13
1.6.2 Pollard's ρ Algorithm . . . 13
1.6.3 Pollard's $p-1$ Algorithm . . . 14
1.6.4 Sieve Methods: Quadratic Sieve and GNFS (Overview) . . . 15
1.6.5 Factoring Records and Key-Size Recommendations . . . 15
1.7 Summary and Bridge to Lesson 2 . . . 17

2 Lesson 2: The RSA Cryptosystem II — Mathematical Attacks **18**

2.1 Wiener's Attack: Small Private Exponent . . . 19
2.1.1 Motivation . . . 19
2.1.2 Continued Fractions Refresher . . . 19
2.1.3 Wiener's Theorem . . . 19
2.2 Håstad's Broadcast Attack . . . 21
2.2.1 Setup . . . 21
2.3 Common Modulus Attack . . . 22
2.3.1 Setup . . . 22
2.4 Cycling Attack (Conceptual) . . . 23
2.4.1 Setup . . . 23
2.5 The Franklin–Reiter Related-Message Attack . . . 24
2.5.1 Setup . . . 24
2.6 Coppersmith's Method: An Overview . . . 25
2.6.1 The General Problem . . . 25
2.6.2 The Reduction to Lattices . . . 25
2.7 Summary and Bridge to Lesson 3 . . . 27

3 Lesson 3: The RSA Cryptosystem III — Implementation and Protocol Attacks **29**
3.1 Padding Oracles: Bleichenbacher's Attack on PKCS#1 v1.5 30
3.1.1 PKCS#1 v1.5 Encryption Padding 30
3.1.2 The Attack, at a High Level 30
3.2 Timing Attacks 32
3.2.1 The Leakage Source 32
3.2.2 Defense: Blinding 32
3.3 Fault Attacks: The Bellcore Attack on CRT-RSA 34
3.3.1 Setup 34
3.4 ROCA: When Key Generation Itself Is the Vulnerability 36
3.4.1 Background 36
3.5 Side-Channel Countermeasures: A Consolidated Checklist 38
3.6 OAEP: A Security Sketch 39
3.6.1 Construction 39
3.7 Guillou–Quisquater: An RSA-Based Zero-Knowledge Proof 41
3.7.1 Motivation: Proving Identity Without Revealing the Secret 41
3.8 Summary 43

Lesson 1: The RSA Cryptosystem I — Foundations, Correctness, and Factoring

Euler's Theorem, the Chinese Remainder Theorem, Key Generation, and Classical Factoring Algorithms

Roadmap

(1) Refresher: Euler's theorem and the Chinese Remainder Theorem (the two facts RSA is built on)

(2) RSA construction: key generation, encryption, decryption

(3) Correctness proof (the full argument, including the edge case textbooks skip)

(4) Security foundations: the RSA problem, the factoring problem, and how they relate

(5) Practical key generation: primality testing, choosing e, avoiding weak primes

(6) Factoring algorithms: trial division, Fermat, Pollard's ρ, Pollard's $p-1$, sieve methods

(7) Key-size recommendations and factoring records

What this lecture deliberately excludes: padding schemes, Bleichenbacher's attack, Wiener's attack, Coppersmith's method, fault attacks. These require the machinery built here and are covered in Lessons 2–3.

1.1 Refresher: The Two Theorems RSA Is Built On

Before defining RSA, we re-derive the two number-theoretic facts it depends on. You have seen these in the math background lectures; we restate them carefully because the correctness proof of RSA (Section 1.3) uses both, and a surprising number of textbook presentations get the general case wrong by ignoring Fact 2.

1.1.1 Euler's Totient Function and Euler's Theorem

Definition 1.1.1 (Euler's totient function)**.** *For $n \in \mathbb{Z}^+$, define*

$$\varphi(n) = |\{a \in \{1, \ldots, n\} : \gcd(a, n) = 1\}|.$$

Proposition 1.1.2 (Multiplicativity and formula)**.** *If $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ is the prime factorization of n, then*

$$\varphi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right).$$

In particular, for two distinct *primes p, q:*

$$\varphi(pq) = (p-1)(q-1).$$

This special case is the one RSA uses directly.

Example 1.1.3 (Computing φ)**.** $36 = 2^2 \cdot 3^2$*, so* $\varphi(36) = 36\left(1 - \frac{1}{2}\right)\left(1 - \frac{1}{3}\right) = 36 \cdot \frac{1}{2} \cdot \frac{2}{3} = 12$*. Direct check:* $\{1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35\}$ *are exactly the 12 residues coprime to 36.*

For the RSA-relevant case: $p = 61, q = 53$ *gives* $\varphi(61 \cdot 53) = 60 \cdot 52 = 3120$ *— this is the pair we will reuse throughout this lecture.*

Theorem 1.1.4 (Euler's Theorem)**.** *If* $\gcd(a, n) = 1$*, then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Proof sketch. The set $(\mathbb{Z}/n\mathbb{Z})^*$ of residues coprime to n forms a group of order $\varphi(n)$ under multiplication mod n. By Lagrange's theorem, for any element a of a finite group G, $a^{|G|} = e$ (the identity). Applying this with $G = (\mathbb{Z}/n\mathbb{Z})^*$ gives $a^{\varphi(n)} \equiv 1 \pmod{n}$. □

Example 1.1.5 (Euler's theorem in action)**.** *Let* $a = 7$*,* $n = 10$*. Since* $\gcd(7, 10) = 1$ *and* $\varphi(10) = \varphi(2)\varphi(5) = 1 \cdot 4 = 4$*, Euler's theorem predicts* $7^4 \equiv 1 \pmod{10}$*. Check:* $7^2 = 49 \equiv 9$*,* $7^4 \equiv 9^2 = 81 \equiv 1 \pmod{10}$*.* ✓

Corollary 1.1.6 (Fermat's Little Theorem)**.** *If p is prime and* $\gcd(a, p) = 1$*, then* $a^{p-1} \equiv 1 \pmod{p}$*. This is Euler's theorem with $n = p$, since* $\varphi(p) = p - 1$*.*

Why RSA cares. Euler's theorem tells us: if we pick an exponent e and a matching d such that $ed \equiv 1 \pmod{\varphi(n)}$, then for any a coprime to n,

$$a^{ed} = a^{1 + k\varphi(n)} = a \cdot (a^{\varphi(n)})^k \equiv a \cdot 1^k \equiv a \pmod{n}.$$

This is *exactly* the decryption-undoes-encryption identity we need. The catch is the phrase "coprime to n" — Section 1.3 deals with what happens when it isn't.

1.1.2 The Chinese Remainder Theorem

Theorem 1.1.7 (CRT)**.** *Let $n = pq$ with p, q coprime. The map*

$$\mathbb{Z}/n\mathbb{Z} \longrightarrow \mathbb{Z}/p\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}, \qquad x \mapsto (x \bmod p,\ x \bmod q)$$

is a ring isomorphism. Equivalently: the system

$$x \equiv a \pmod{p}, \qquad x \equiv b \pmod{q}$$

has a unique solution $x \in \{0, \ldots, n-1\}$ for every pair (a, b).

Proof sketch. The map is a ring homomorphism (checking addition and multiplication are preserved is routine). It is injective because if $x \equiv y \pmod{p}$ and $x \equiv y \pmod{q}$ then $pq \mid (x - y)$ (using $\gcd(p, q) = 1$), so $x = y$ in $\mathbb{Z}/n\mathbb{Z}$. Since both sides have exactly $n = pq$ elements, injective implies bijective. □

Example 1.1.8 (Solving a CRT system by hand)**.** *Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$. Listing multiples: $x \in \{2, 5, 8, 11, 14, \ldots\}$ satisfies the first; checking against the second, $x = 8$ gives $8 \equiv 3 \pmod{5}$. ✓ So $x \equiv 8 \pmod{15}$, and this is the unique solution mod 15 guaranteed by the theorem. This toy mechanism — solve mod each prime separately, recombine — is exactly what CRT-RSA decryption automates at scale (Section 1.2.3).*

Remark 1.1.9 (Two uses of CRT in this lecture)**.** *CRT plays two completely different roles in RSA that students often conflate:*

1. ***Correctness**: it lets us prove decryption works for* all *messages $m \in \{0, \ldots, n-1\}$, not just those coprime to n (Section 1.3).*
2. ***Efficiency**: it lets us speed up decryption by roughly $4\times$ using CRT-RSA (Section 1.2.3) — and this optimization is precisely what makes the Bellcore fault attack possible, which we cover in Lesson 3.*

1.2 RSA Construction

1.2.1 Key Generation

Algorithm 1 RSA Key Generation

1: Choose two large distinct primes p, q of similar bit-length (typically 1536 bits each for a 3072-bit modulus)
2: Compute $n = pq$
3: Compute $\varphi(n) = (p-1)(q-1)$
4: Choose e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$
5: Compute $d \equiv e^{-1} \pmod{\varphi(n)}$ via the Extended Euclidean Algorithm
6: **Public key:** (n, e) **Private key:** (n, d) (or (p, q, d) for CRT speedup)

Definition 1.2.1 (RSA Encryption / Decryption)**.** *For a message $m \in \{0, 1, \ldots, n-1\}$:*

$$\textit{Encryption:} \quad c = m^e \bmod n, \qquad \textit{Decryption:} \quad m = c^d \bmod n.$$

Remark 1.2.2 (Why $\gcd(e, \varphi(n)) = 1$ is required)**.** *We need e to be* invertible *modulo $\varphi(n)$, i.e. we need some d with $ed \equiv 1 \pmod{\varphi(n)}$. An element of $\mathbb{Z}/\varphi(n)\mathbb{Z}$ is invertible iff it is coprime to $\varphi(n)$ (standard fact from group theory: $(\mathbb{Z}/k\mathbb{Z})^*$ consists exactly of residues coprime to k). Without this, no valid d exists at all.*

Example 1.2.3 (Toy example — never use these sizes in practice)**.** *Let $p = 61$, $q = 53$. Then $n = 3233$, $\varphi(n) = 60 \cdot 52 = 3120$. Choose $e = 17$ (check $\gcd(17, 3120) = 1$). Extended Euclid gives $d = 2753$, since $17 \times 2753 = 46801 = 15 \times 3120 + 1$. Encrypt $m = 65$: $c = 65^{17} \bmod 3233 = 2790$. Decrypt: $2790^{2753} \bmod 3233 = 65$.* ✓

1.2.2 Fast Modular Exponentiation

Both encryption and decryption require computing $a^k \bmod n$ efficiently. Naive repeated multiplication is $O(k)$ multiplications; **square-and-multiply** reduces this to $O(\log k)$.

Algorithm 2 Square-and-Multiply (Right-to-Left)

```
function ModExp(a, k, n)
    result ← 1
    base ← a mod n
    while k > 0 do
        if k mod 2 = 1 then
            result ← (result × base) mod n
        end if
        base ← (base × base) mod n
        k ← ⌊k/2⌋
    end while
    return result
end function
```

Example 1.2.4 (Square-and-multiply trace)**.** *Compute $5^{117} \bmod 19$. Write $117 = 1110101_2$*

(bits, LSB first: $1, 0, 1, 0, 1, 1, 1$*). Trace of* ModExp*, showing* (*bit consumed*, *result*, *base after squaring*)*:*

step	*bit*	*result*	*base*
1	1	5	6
2	0	5	17
3	1	9	4
4	0	9	16
5	1	11	9
6	1	4	5
7	1	1	6

Final result: 1*. Direct check:* $5^{117} \bmod 19 = 1$ *(consistent, since* $\mathrm{ord}_{19}(5) \mid \varphi(19) = 18$*, and indeed* $117 \equiv 9 \pmod{18}$ *while* $5^9 \equiv 1 \pmod{19}$ *— try verifying this independently). Only* ***7 squarings and 5 multiplications*** *were needed instead of 116 successive multiplications.*

Remark 1.2.5 (Security-relevant note — preview of Lesson 3)**.** *The naive implementation above has data-dependent branching (the* `if` *depends on the secret exponent's bits), which leaks timing information. This is the root cause of timing attacks on RSA, covered when we do implementation attacks.*

1.2.3 CRT-Based Decryption Speedup

Since the private-key holder knows p and q, decryption can be done mod p and mod q separately (smaller exponents, smaller moduli) and recombined via CRT.

Algorithm 3 CRT-RSA Decryption

Precompute: $d_p = d \bmod (p-1), \quad d_q = d \bmod (q-1), \quad q_{inv} = q^{-1} \bmod p$
$m_p = c^{d_p} \bmod p$
$m_q = c^{d_q} \bmod q$
$h = q_{inv} \cdot (m_p - m_q) \bmod p$
$m = m_q + h \cdot q$
return m

Example 1.2.6 (CRT-RSA decryption, continuing the toy example)**.** *Recall* $p = 61, q = 53, n = 3233, e = 17, d = 2753$*, and* $c = 2790$ *(encryption of* $m = 65$ *from the earlier example). Precompute:*

$$d_p = d \bmod (p-1) = 2753 \bmod 60 = 53, \qquad d_q = d \bmod (q-1) = 2753 \bmod 52 = 49, \qquad q_{inv} = q^{-1} \bmod p = 3$$

Then:

$$m_p = c^{d_p} \bmod p = 2790^{53} \bmod 61 = 4, \qquad m_q = c^{d_q} \bmod q = 2790^{49} \bmod 53 = 12.$$

Recombine: $h = q_{inv}(m_p - m_q) \bmod p = 38 \cdot (4 - 12) \bmod 61 = 1$*, and* $m = m_q + h \cdot q = 12 + 1 \cdot 53 = 65$*.* ✓*Matches the direct decryption, but every intermediate exponentiation used a* $\sim$*6-bit exponent and a* $\sim$*6-bit modulus instead of the full-size* d *and* n*.*

This gives roughly a 4× speedup (two exponentiations with half-size exponents and half-size moduli, each individually $\sim 8\times$ cheaper, combined with cheap recombination). **Flag this in your memory**: this optimization is exactly what the Bellcore fault attack (Lesson 3) exploits — a single computational fault in either the m_p or m_q branch lets an attacker factor n from one faulty signature.

1.3 Correctness of RSA

We now prove $\mathrm{Dec}(\mathrm{Enc}(m)) = m$ for *every* $m \in \{0, \ldots, n-1\}$ — not just those coprime to n. Most textbooks quietly assume $\gcd(m, n) = 1$ and invoke Euler's theorem directly; a rigorous treatment must handle the small-probability edge case, and this is exactly where CRT earns its keep.

Theorem 1.3.1 (RSA Correctness)**.** *For all* $m \in \{0, \ldots, n-1\}$, $\quad m^{ed} \equiv m \pmod{n}$.

Proof. Since $ed \equiv 1 \pmod{\varphi(n)}$, write $ed = 1 + k\varphi(n) = 1 + k(p-1)(q-1)$ for some integer $k \geq 0$.

By CRT (Theorem 1.1.7), it suffices to show

$$m^{ed} \equiv m \pmod{p} \qquad \text{and} \qquad m^{ed} \equiv m \pmod{q},$$

since these together are equivalent to $m^{ed} \equiv m \pmod{pq}$.

We show the first congruence; the second is symmetric. Consider two cases.

Case 1: $p \nmid m$**.** Then $\gcd(m, p) = 1$, so by Fermat's Little Theorem $m^{p-1} \equiv 1 \pmod{p}$. Hence

$$m^{ed} = m^{1+k(p-1)(q-1)} = m \cdot \left(m^{p-1}\right)^{k(q-1)} \equiv m \cdot 1^{k(q-1)} \equiv m \pmod{p}.$$

Case 2: $p \mid m$**.** Then $m \equiv 0 \pmod{p}$, and since $ed \geq 1$, $m^{ed} \equiv 0 \equiv m \pmod{p}$ trivially.

Either way, $m^{ed} \equiv m \pmod{p}$. By the symmetric argument, $m^{ed} \equiv m \pmod{q}$. By CRT, $m^{ed} \equiv m \pmod{pq} = \pmod{n}$. □

Remark 1.3.2 (Why this edge case matters pedagogically, not just practically)**.** *The event* $p \mid m$ *for a random* $m \in \{0, \ldots, n-1\}$ *has probability* $1/p$ *— utterly negligible for cryptographic-size primes, so it is* never *a practical concern. But it matters conceptually: it shows correctness of RSA is really a CRT argument dressed up, and it is the reason the correctness proof cannot be "just Euler's theorem" as often claimed. This distinction resurfaces when we discuss why RSA's algebraic structure (a ring homomorphism argument, not merely a group-order argument) enables attacks like Håstad's broadcast attack in Lesson 2.*

Example 1.3.3 (The edge case, made concrete)**.** *With* $n = 3233 = 61 \times 53$*, the "bad" messages are the tiny fraction that are multiples of* 61 *or* 53*: e.g.,* $m = 61$*. Here* $p \mid m$*, so Case 2 applies mod* 61 *(*$m^{ed} \equiv 0 \equiv m \pmod{61}$ *trivially), while Case 1 applies mod* 53 *(since* $\gcd(61, 53) = 1$*, Fermat's little theorem does the work mod* 53*). CRT stitches the two together. You can verify numerically that* $61^{17 \cdot 2753} \bmod 3233 = 61$ *still holds — correctness survives even though Euler's theorem alone (which needs* $\gcd(m, n) = 1$*) does not directly apply to* $m = 61$*, since* $\gcd(61, 3233) = 61 \neq 1$*.*

Exercise 1.3.4. *Prove the case* $q \mid m$ *directly (mirror the argument above), and explain why* $p \mid m$ *and* $q \mid m$ *cannot both hold when* $m < n = pq$ *and* $m \neq 0$*.*

1.4 Security Foundations

1.4.1 The RSA Problem and the RSA Assumption

Definition 1.4.1 (RSA Problem)**.** *Given* (n, e) *and* $c = m^e \bmod n$ *for a uniformly random* $m \in \mathbb{Z}_n^*$*, compute* m*.*

Definition 1.4.2 (RSA Assumption)**.** *No probabilistic polynomial-time algorithm solves the RSA problem with non-negligible probability (over the choice of* p, q, e, m*), for suitably generated* n*.*

The critical logical relationship.

$$\text{Factoring } n \implies \text{Breaking RSA}, \qquad \text{but the converse is } \textit{not known} \text{ to hold.}$$

If you can factor $n = pq$, you immediately recover $\varphi(n) = (p-1)(q-1)$, compute $d = e^{-1} \bmod \varphi(n)$, and decrypt anything. So **Factoring** $\leq_p$ **Breaking RSA** (factoring reduces to breaking RSA — breaking RSA is at least as hard as factoring, in the sense that an efficient factoring algorithm gives an efficient RSA-breaking algorithm).

The open question, since 1977, is whether the reduction goes the other way: does an oracle that breaks RSA (computes e-th roots mod n) let you factor n? This is **not known in general**. There exist special/weakened settings (e.g., Rabin's cryptosystem, and RSA with a specific low-exponent oracle in certain models) where equivalence to factoring *can* be shown, but for standard RSA the assumption is a genuinely separate, slightly stronger-looking, hardness assumption than "factoring is hard."

Remark 1.4.3 (Why this matters for how we teach attacks)**.** *Every specific attack technique in Lessons 2–3 (Wiener, Coppersmith, Håstad, Bleichenbacher, timing/fault attacks) breaks RSA* without factoring n*. This is the point: they exploit structural weaknesses in* how RSA is used *(small* d*, small* e *with related messages, malleable padding, faulty computation) rather than solving the underlying hard problem. Keep this distinction sharp for students: "RSA is broken" almost always means "this particular use of RSA is broken," not "factoring got easy."*

1.4.2 Why Textbook RSA Is Not Secure As-Is

Even setting aside implementation attacks, **textbook RSA** (encrypting $m^e \bmod n$ directly, with no padding) has structural problems worth flagging now, to motivate why Lesson 3 exists at all:

- **Deterministic**: the same plaintext always produces the same ciphertext under a given key ⇒ no semantic security; an attacker can build a dictionary of likely plaintexts and their ciphertexts.
- **Malleable**: given $c = m^e \bmod n$, anyone can compute $c' = (2^e \cdot c) \bmod n = (2m)^e \bmod n$, a valid encryption of $2m$, without knowing m. No integrity guarantee whatsoever.
- **Small-message / small-exponent issues**: if $m^e < n$ (no modular reduction occurs), $c = m^e$ over the integers, and m is recovered by an ordinary integer e-th root — no cryptography needed. This is the seed of Håstad's broadcast attack (Lesson 2).

Example 1.4.4 (Malleability, worked)**.** *Using* $n = 3233, e = 17, d = 2753$ *and* $c = 2790$ *(encryption of* $m = 65$*): compute* $c' = (2^{17} \bmod 3233) \cdot c \bmod 3233 = 3017$*. Decrypting,* $c'^d \bmod n = 130 = 2 \times 65$*. An attacker who intercepts* c *and wants to double the plaintext (e.g., doubling a monetary amount in a naive protocol) needs* no key material at all *to do this.*

Example 1.4.5 (No modular wraparound, worked)**.** *Let* $p = 17, q = 19$*, so* $n = 323$*, and suppose* $e = 3$ *(small exponent, no padding). Encrypt* $m = 5$*:* $c = 5^3 = 125$*. Since* $125 < 323$*,* no reduction mod n ever occurred — c *is just the integer* 125*. An eavesdropper who does not know* d*,* p*, or* q *recovers* m *by taking an ordinary integer cube root:* $\sqrt[3]{125} = 5$*. This is not a flaw in the mathematics of RSA — it is a flaw in using RSA* without padding *on small messages with small* e*, and it generalizes (via CRT, not integer roots) to Håstad's broadcast attack in Lesson 2, where the message need not literally satisfy* $m^e < n$*.*

This is why real systems use **padded RSA** (PKCS#1 v1.5, OAEP) — covered in Lesson 3 alongside the attacks that padding is designed to prevent, and the attacks that emerged *against* specific padding schemes (Bleichenbacher).

1.5 Practical Key Generation

1.5.1 Primality Testing

Key generation needs to sample large primes. We cannot test primality by trial division for 1536-bit numbers, so we use **probabilistic primality tests**.

Miller–Rabin Primality Test (statement only — proof is number-theoretic and out of scope here). Write $n-1 = 2^s d$ with d odd. Pick random witnesses $a \in \{2, \ldots, n-2\}$ and check whether

$$a^d \equiv 1 \pmod{n} \quad \text{or} \quad a^{2^r d} \equiv -1 \pmod{n} \text{ for some } 0 \le r < s.$$

If neither holds, n is *composite* (certain). If it holds, n is *probably prime*: the test has false-positive probability $\le 1/4$ per random witness, so k independent witnesses give error probability $\le 4^{-k}$. In practice $k \approx 40$ is used for cryptographic key generation, giving error probability $< 2^{-80}$.

Example 1.5.1 (Miller–Rabin, worked — and why one witness isn't enough)**.** *Let $n = 91 = 7 \times 13$ (composite). Write $n - 1 = 90 = 2^1 \cdot 45$, so $s = 1, d = 45$.*

***Witness** $a = 3$**:** $3^{45} \bmod 91 = 27$. This is neither 1 nor $-1 \equiv 90 \pmod{91}$, and there are no further squarings to check ($s = 1$ means only $r = 0$ is available). So $a = 3$* correctly exposes *91 as composite.*

***Witness** $a = 10$**:** $10^{45} \bmod 91 = 90 \equiv -1 \pmod{91}$. This* passes *the test — $a = 10$ is a false witness for compositeness here, illustrating exactly why the test is only probabilistic and why cryptographic key generation uses dozens of independent random witnesses rather than one.*

1.5.2 Choosing e and d

- **Common choice**: $e = 65537 = 2^{16} + 1$. It is prime, has a Hamming weight of 2 (fast exponentiation: only 17 squarings + 1 multiplication), and is large enough to avoid the naive small-e pitfalls above.
- **Avoid** $e = 3$ without proper padding — historically responsible for several real-world broadcast-style vulnerabilities (Lesson 2).
- d must be **large** — roughly the same bit-length as $\varphi(n)$. A small d (say $d < n^{0.25}$, roughly) is catastrophic: Wiener's continued-fraction attack recovers it directly from (n, e) alone. Full derivation in Lesson 2.

1.5.3 Prime Generation Pitfalls

Naively generated primes can be individually "strong" (large, probably-prime) yet jointly **weak** if not generated with enough entropy or independence:

- **Shared-prime attacks**: if two different RSA moduli $n_1 = pq_1$ and $n_2 = pq_2$ share a prime factor p (due to poor entropy at key-generation time — this happened in real embedded devices), then $\gcd(n_1, n_2) = p$ instantly, computable in polynomial time via Euclid's algorithm even across a database of millions of public keys. This was a real, large-scale finding (Heninger et al., 2012; Lenstra et al., 2012) affecting embedded/IoT devices with weak randomness.

  Example 1.5.2 (Shared-prime attack, worked)**.** *Suppose (toy sizes) two devices generate $n_1 = p \cdot q_1$ and $n_2 = p \cdot q_2$ with a shared factor $p = 1000003$ due to a weak entropy pool at boot time, where $q_1 = 1009$, $q_2 = 2003$. Then $n_1 = 1009003027$ and $n_2 = 2003006009$. An*

attacker with no other information *computes* $\gcd(n_1, n_2) = 1000003 = p$ *via the Euclidean algorithm in microseconds, instantly factoring* both *moduli. At internet scale, this is done pairwise (or with a batch-gcd algorithm) across millions of collected public keys — no individual modulus looks weak in isolation, but shared factors across the population are what break.*

- **Smooth** $p-1$: if $p-1$ has only small prime factors, p is vulnerable to Pollard's $p-1$ method (Section 1.6.3). Historically "strong primes" (where $p-1$ has a large prime factor) were recommended as a defense; modern guidance considers this largely unnecessary if primes are simply chosen large and random, since the probability of accidentally-smooth $p-1$ is negligible at current key sizes — but it is worth knowing *why* the old advice existed.

1.6 Factoring Algorithms

The entire security of RSA is calibrated against the best known classical factoring algorithms. This section builds the toolkit needed to justify (and periodically revise) key-size recommendations.

1.6.1 Trial Division and Fermat's Method

Trial division: try all primes up to $\sqrt{n}$. Cost $O(\sqrt{n})$ — completely infeasible for cryptographic n (a 2048-bit n has $\sqrt{n} \approx 2^{1024}$).

Fermat's Factorization Method. If $n = pq$ with p, q close together, write $n = a^2 - b^2 = (a-b)(a+b)$ where $a = \frac{p+q}{2}$, $b = \frac{p-q}{2}$. Search for a starting at $\lceil \sqrt{n} \rceil$, checking if $a^2 - n$ is a perfect square. If p, q differ by only a few bits, this converges almost immediately.

Example 1.6.1 (Fermat's method, worked)**.** *Let $p = 10007$, $q = 10009$ (deliberately close). Then $n = pq = 100160063$, and $\lceil \sqrt{n} \rceil = 10008$. Try $a = 10008$: $a^2 - n = 10008^2 - 100160063 = 1 = 1^2$, a perfect square with $b = 1$. Immediately, $p = a - b = 10007$ and $q = a + b = 10009$. **One single trial** recovered the full factorization — this is why close primes are catastrophic, regardless of how large n itself is.*

Practical consequence. Never choose p and q close to each other (e.g., both near $\sqrt{n}$ with only a few differing bits) — Fermat's method factors such n almost instantly. This is why key-generation standards mandate p, q be independently random over their full bit-length range, not merely "two large primes."

1.6.2 Pollard's ρ Algorithm

A far better generic method, based on a birthday-paradox argument, with expected running time $O(n^{1/4})$ (i.e., $O(\sqrt{p})$ where p is the smallest prime factor).

Algorithm 4 Pollard's ρ (Floyd cycle-detection variant)

Pick pseudo-random function $f(x) = x^2 + c \bmod n$ (some small constant $c \neq 0, -2$)
$x \leftarrow 2$, $y \leftarrow 2$
repeat
    $x \leftarrow f(x)$
    $y \leftarrow f(f(y))$
    $d \leftarrow \gcd(|x - y|, n)$
until $d \neq 1$
if $d = n$ **then**
    Restart with a different f
else
    return d (a nontrivial factor)
end if

Heuristic argument for why this works. Treat f as a pseudo-random function on $\mathbb{Z}/p\mathbb{Z}$ (for p the unknown smallest factor). By the birthday paradox, the sequence $x_0, x_1, x_2, \ldots \pmod{p}$ collides (enters a cycle) after roughly $O(\sqrt{p})$ steps, even though the same sequence mod n does not yet show any visible structure (since n is much larger). Once $x_i \equiv x_j \pmod{p}$ but $x_i \neq x_j \pmod{n}$, we get $p \mid (x_i - x_j)$ but $n \nmid (x_i - x_j)$, so $\gcd(x_i - x_j, n)$ reveals p (or a multiple structure containing it) without ever knowing p in advance. Floyd's "tortoise and hare" technique detects this collision using $O(1)$ memory.

Example 1.6.2 (Pollard's ρ, worked)**.** *Let* $n = 10403 = 101 \times 103$*. Using* $f(x) = x^2 + 1 \bmod n$*, starting* $x_0 = y_0 = 2$*:*

step	x	y	$\gcd(\lvert x - y\rvert, n)$
1	5	26	1
2	26	598	1
3	677	3418	1
4	598	3531	1
5	3903	3724	1
6	3418	9812	1
7	156	9970	1
8	3531	3682	1
9	5168	7087	**101**

After just 9 iterations, a nontrivial factor 101 *appears — versus* 101 *trial divisions (or* $\sqrt{101} \approx 10$ *under* ρ*'s* $O(\sqrt{p})$ *bound, matching what we observe) to find the same factor by brute force on a number this small. The gap widens enormously as* n *grows.*

Relevance to RSA: Pollard's ρ is why RSA moduli need *both* prime factors to be large — it factors n in time proportional to $\sqrt{p_{\min}}$, the smallest factor, so an unbalanced $n = pq$ with a small p is broken quickly regardless of how large q is or how large n overall appears.

1.6.3 Pollard's $p - 1$ Algorithm

Exploits primes p where $p - 1$ is **smooth** (product of only small primes).

Algorithm 5 Pollard's $p - 1$

```
Choose smoothness bound B
a ← 2
for each prime ℓ ≤ B do
    a ← a^(ℓ^⌊log_ℓ B⌋) mod n
end for
d ← gcd(a − 1, n)
if 1 < d < n then
    return d
end if
```

Why it works: if $p-1$ divides $M = \operatorname{lcm}(1, 2, \ldots, B)$ (all its prime power factors are $\leq B$), then by Fermat's little theorem $a^M \equiv 1 \pmod{p}$ for any a coprime to p, so $p \mid \gcd(a^M - 1, n)$, revealing p — while $q - 1$ (assumed non-smooth) does not similarly divide M, so $q \nmid (a^M - 1)$ typically, and the gcd isolates p cleanly.

Defense: choose p such that $p-1$ has at least one large prime factor ("strong prime" criterion, historically recommended; largely moot at 2048+ bit sizes chosen with proper randomness, but essential to understand conceptually).

Example 1.6.3 (Pollard's $p - 1$, worked)**.** *Let* $p = 1009$ *(so* $p - 1 = 1008 = 2^4 \cdot 3^2 \cdot 7$*, smooth) and* $q = 2027$ *(so* $q - 1 = 2026 = 2 \cdot 1013$*,* not *smooth — 1013 is a large prime). Then* $n = pq = 2045243$*.*

Choose smoothness bound $B = 16$ *(large enough that* $2^4, 3^2, 7 \leq 16$ *are all captured). Build* $M = \prod_{\ell \leq B} \ell^{\lfloor \log_\ell 16 \rfloor} = 2^4 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 13 = 720720$*, which is divisible by* 1008*. Compute* $a = 2^M \bmod n = 2024055$*, then* $\gcd(a - 1, n) = \mathbf{1009} = p$*. The attack succeeds because every prime power dividing* $p - 1$ *also divides* M*, so* $2^M \equiv 1 \pmod{p}$ *(Fermat), while* $q - 1 = 2 \cdot 1013$ *does* not *divide* M *(since* $1013 > B$*), so no such collapse happens mod* q *— the gcd cleanly isolates* p*.*

***Contrast**: attempting the same procedure with $B = 7$ (too small) fails — $\gcd(2^{M'}-1, n) = 1$ for the smaller $M' = 420$, since $2^4 = 16 \nmid 420$ even though $2 \mid 420$; the bound must be large enough to capture the* full power *of each small prime dividing $p-1$, not just its presence.*

1.6.4 Sieve Methods: Quadratic Sieve and GNFS (Overview)

These are the algorithms that actually matter for modern RSA key-size decisions — Pollard's methods are pedagogically important but are not the bottleneck for well-generated 2048+ bit RSA keys.

The core idea shared by all sieve-based factoring (Fermat, QS, GNFS). Find integers x, y with $x^2 \equiv y^2 \pmod{n}$ but $x \not\equiv \pm y \pmod{n}$. Then $n \mid (x-y)(x+y)$ but $n \nmid (x-y)$ and $n \nmid (x+y)$, so $\gcd(x-y, n)$ yields a nontrivial factor. The entire game is finding such x, y efficiently — this is what distinguishes Fermat's method (too slow to find them) from the Quadratic Sieve (clever, systematic search using smooth numbers and linear algebra over $\mathbb{F}_2$) from the General Number Field Sieve (same idea, but working in a number field for asymptotically better complexity).

Example 1.6.4 (The core relation, worked on a tiny modulus)**.** *Let $n = 35$. Notice $6^2 = 36 \equiv 1 \equiv 1^2 \pmod{35}$, so $x = 6, y = 1$ satisfies $x^2 \equiv y^2 \pmod{n}$. Check $x \not\equiv \pm y \pmod{35}$: indeed $6 \neq 1$ and $6 \neq -1 \equiv 34$. Then $\gcd(x-y, n) = \gcd(5, 35) = 5$ and $\gcd(x+y, n) = \gcd(7, 35) = 7$ — both nontrivial factors, recovering $35 = 5 \times 7$ instantly. QS and GNFS scale this exact idea to numbers with hundreds of digits by systematically manufacturing such (x, y) pairs from products of "smooth" relations, then solving a large linear system over $\mathbb{F}_2$ to combine them — full derivation is beyond this course, but the underlying trick is precisely this one identity.*

Algorithm	**Heuristic complexity**	**Practical relevance**
Trial division	$O(\sqrt{n})$	Never, for crypto-size n
Fermat's method	Fast only if $p \approx q$	Defense: choose p, q independently random
Pollard's ρ	$O(n^{1/4})$	Breaks unbalanced factorizations
Pollard's $p-1$	Fast if $p-1$ smooth	Defense: avoid smooth $p-1$ (minor concern today)
Quadratic Sieve	$L_n[1/2, 1] \approx e^{(1+o(1))\sqrt{\ln n \ln\ln n}}$	Best for n up to ∼100 digits
GNFS	$L_n[1/3, (64/9)^{1/3}]$	Best known for large RSA moduli

Remark 1.6.5 (L-notation)**.** $L_n[\alpha, c] = \exp\left(c(\ln n)^\alpha (\ln\ln n)^{1-\alpha}\right)$ *interpolates between polynomial time ($\alpha = 0$) and exponential time ($\alpha = 1$) in the bit-length of n. GNFS's $\alpha = 1/3$ is* ***sub-exponential*** *— much better than exponential, but still infeasible at current RSA key sizes, which is exactly why 2048/3072-bit RSA remains (classically) secure.*

1.6.5 Factoring Records and Key-Size Recommendations

Modulus size	**Status (as of most recent public records)**	**Recommendation**
512 bits	Factored in hours on cloud hardware	Broken, never use
768 bits	Factored (RSA-768, 2009-era GNFS effort)	Broken
829 bits (RSA-250)	Factored (2020)	Broken
1024 bits	Not publicly factored, but weak margin	Deprecated, phase out
2048 bits	No known feasible classical attack	Current baseline minimum
3072 bits	No known feasible classical attack	Recommended for long-term security
4096 bits	No known feasible classical attack	Used for high-assurance/long-lived ke

Remark 1.6.6 (Quantum caveat)**.** *Shor's algorithm factors n in polynomial time on a sufficiently large fault-tolerant quantum computer, breaking RSA at* any *classical key size. This is precisely the motivation for lattice-based (post-quantum) cryptography: lattice problems (like LWE/SVP) have no known efficient quantum algorithm, unlike factoring and discrete log.*

1.7 Summary and Bridge to Lesson 2

Summary

- RSA's correctness rests on Euler's theorem *and* CRT together — not Euler's theorem alone.
- Security rests on the RSA assumption, which is only known to be *implied by* hardness of factoring, not equivalent to it.
- Textbook (unpadded) RSA is deterministic and malleable — insecure regardless of key size.
- Practical key generation requires care: large random independent primes, avoiding smooth $p-1$, avoiding small/related moduli across devices.
- The best classical factoring algorithm (GNFS) is sub-exponential, which is why 2048+ bit keys remain safe classically, and why quantum algorithms (Shor's) are an existential threat requiring post-quantum alternatives.

Looking Ahead — Lesson 2 — Mathematical Attacks on RSA

Next lecture assumes n itself is *not* factored, and instead breaks RSA through misuse:

- **Wiener's attack**: recovers d directly from (n, e) when d is too small, via continued fraction expansion of e/n.
- **Håstad's broadcast attack**: recovers m when the same message is sent to multiple recipients with small e, via CRT (yes — CRT again, but now as an attack tool).
- **Coppersmith's method**: recovers partial/short plaintexts or exposed key bits using lattice-based root-finding (a nice callback to lattice cryptography, if you have covered it).
- **Common modulus attack**: breaks RSA when two parties share n but use different e.

Bring: Extended Euclidean Algorithm and CRT fresh in memory — both get reused, this time as attack tools rather than construction tools.

Tutorial Exercises

1. Complete the proof of RSA correctness for the case $q \mid m$.
2. Given $p = 17$, $q = 23$, $e = 7$: compute n, $\varphi(n)$, and d. Encrypt $m = 41$ and verify decryption recovers it.
3. Explain, in your own words, why $\gcd(e, \varphi(n)) = 1$ is necessary but e being prime is not.
4. Run Pollard's ρ by hand on $n = 8051$ using $f(x) = x^2 + 1$, starting at $x_0 = 2$. Find a nontrivial factor.
5. Suppose an RSA key-generation implementation always picks q as the next prime after p (i.e., $q = \text{nextprime}(p)$). Explain concretely how you would factor such a modulus, and estimate the cost.
6. Research question: look up the RSA-250 factorization (2020) and report what algorithm was used, and how much compute time it took. Compare against a naive trial-division estimate for the same modulus size.

Lesson 2: The RSA Cryptosystem II — Mathematical Attacks

Wiener's Attack, Håstad's Broadcast Attack, Common Modulus Attacks, Franklin–Reiter, and Coppersmith's Method

Roadmap

(1) Wiener's Attack: recovering a small private exponent d from (n, e) alone, via continued fractions

(2) Håstad's Broadcast Attack: recovering m from low-exponent encryptions to multiple recipients, via CRT

(3) Common Modulus Attack: breaking RSA when two parties share n but use different, coprime exponents

(4) Cycling Attack: a conceptual (impractical) attack via the multiplicative order of c

(5) The Franklin–Reiter Related-Message Attack: breaking two related plaintexts under the same low exponent, via polynomial GCD

(6) Coppersmith's Method: an overview of small-root finding via lattice reduction (the LLL algorithm)

What every attack in this lecture has in common: none of them factor n. Each exploits a specific way RSA was *used* — a too-small d, a too-small e combined with related or repeated messages, exponent/modulus reuse, or the sheer algebraic structure of modular exponentiation. This is the point made in Lesson 1: breaking RSA is not the same as solving the RSA problem in general, and it is very much not the same as factoring.

2.1 Wiener's Attack: Small Private Exponent

2.1.1 Motivation

CRT-RSA decryption (Lesson 1, Section 1.2.3) is faster with smaller d. This creates a temptation: why not simply choose a small d to speed up decryption, and derive $e = d^{-1} \bmod \varphi(n)$ afterward? Wiener showed in 1990 that this is catastrophic.

2.1.2 Continued Fractions Refresher

Definition 2.1.1 (Continued fraction expansion)**.** *Every rational number $\frac{x}{y}$ has a unique finite continued fraction expansion*

$$\frac{x}{y} = a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{\ddots + \cfrac{1}{a_k}}}} =: [a_0; a_1, a_2, \ldots, a_k],$$

computed by the Euclidean algorithm: $a_0 = \lfloor x/y \rfloor$, then recurse on $y/(x - a_0 y)$.

Definition 2.1.2 (Convergents)**.** *The i-th **convergent** $\frac{h_i}{k_i}$ of $[a_0; a_1, \ldots]$ is the value of the truncated expansion $[a_0; a_1, \ldots, a_i]$, computed by the recurrence*

$$h_i = a_i h_{i-1} + h_{i-2}, \qquad k_i = a_i k_{i-1} + k_{i-2}, \qquad (h_{-1}{=}1, h_{-2}{=}0,\ k_{-1}{=}0, k_{-2}{=}1).$$

Theorem 2.1.3 (Legendre's theorem on best rational approximations)**.** *If $\left|\frac{x}{y} - \frac{h}{k}\right| < \frac{1}{2k^2}$, then $\frac{h}{k}$ is a convergent of the continued fraction expansion of $\frac{x}{y}$.*

Why this matters for RSA. This theorem is an *efficient search tool*: instead of guessing $\frac{h}{k}$ among astronomically many fractions, if we know $\frac{h}{k}$ is a sufficiently good rational approximation of some known quantity $\frac{x}{y}$, it is *guaranteed* to appear among the (only $O(\log y)$ many) convergents — which we can compute quickly via the Euclidean algorithm.

2.1.3 Wiener's Theorem

Theorem 2.1.4 (Wiener, 1990)**.** *Let $n = pq$ with $q < p < 2q$, and let $d < \frac{1}{3}n^{1/4}$. Then given only the public key (n, e) with $ed \equiv 1 \pmod{\varphi(n)}$, d can be recovered in polynomial time.*

Proof idea. Since $ed \equiv 1 \pmod{\varphi(n)}$, write $ed = 1 + k\varphi(n)$ for some positive integer $k < d$ (one can show $k < d$ using $e < \varphi(n)$). Then

$$\left|\frac{e}{n} - \frac{k}{d}\right| = \left|\frac{ed - kn}{nd}\right| = \left|\frac{1 + k\varphi(n) - kn}{nd}\right| = \frac{k(n - \varphi(n)) - 1}{nd}.$$

Using $n - \varphi(n) = p + q - 1 < 3\sqrt{n}$ (a consequence of $q < p < 2q$) and $k < d$, this quantity can be bounded above by $\frac{1}{2d^2}$ whenever $d < \frac{1}{3}n^{1/4}$. By Legendre's theorem, $\frac{k}{d}$ therefore appears as *some* convergent of the continued fraction expansion of $\frac{e}{n}$ — a quantity the attacker already knows in full, from the public key alone. □

Example 2.1.5 (Wiener's attack, fully worked)**.** *Let $p = 1223$, $q = 1987$, so $n = 2{,}430{,}101$ and $\varphi(n) = 2{,}426{,}892$. Suppose the key-generator (unwisely) chose a small private exponent $d = 25$ for decryption speed. Then $e = d^{-1} \bmod \varphi(n) = 2{,}135{,}665$ — this is the only thing published.*

Algorithm 6 Wiener's Attack

Compute the continued fraction expansion of e/n
Compute all convergents h_i/k_i
for each convergent k/d_{cand} **do**
    if $k \neq 0$ and $(e \cdot d_{\text{cand}} - 1) \bmod k = 0$ **then**
        $\varphi_{\text{cand}} \leftarrow (e \cdot d_{\text{cand}} - 1)/k$
        Solve $x^2 - (n - \varphi_{\text{cand}} + 1)x + n = 0$ for integer roots p, q
        if integer roots exist and $pq = n$ **then**
            return $d = d_{\text{cand}}$ (attack succeeded)
        end if
    end if
end for

Check the bound: $\frac{1}{3}n^{1/4} \approx 13.16$. *Wait —* $d = 25 > 13.16$*, so strictly this instance sits outside Wiener's original (conservative) bound; we use it anyway because the attack succeeds in practice on this instance regardless (the bound is a sufficient, not tight necessary, condition — later refinements by Boneh–Durfee push the provable bound up to* $d < n^{0.292}$*, which comfortably covers this example). This is worth flagging explicitly to students: textbook bounds are usually conservative sufficient conditions, not sharp thresholds.*

Run the attack: *the continued fraction expansion of* $e/n = 2135665/2430101$ *is*

$$[0; 1, 7, 3, 1, 17, 1, 1, 2, 1, 2, 211].$$

Its convergents h_i/k_i *include (among others)* $\frac{22}{25}$*. Testing* $k = 22, d_{cand} = 25$*:* $(e \cdot 25 - 1) \bmod 22 = 0$*, giving* $\varphi_{cand} = 2{,}426{,}892$ *— exactly right. Solving the resulting quadratic for* p, q *recovers* $p = 1223, q = 1987$ ***exactly****. The attacker recovered the* entire private key *— not just* d*, but the factorization of* n *itself — using only the public key* (n, e) *and a handful of Euclidean-algorithm steps.*

Practical consequence. Never choose d for efficiency and derive e afterward. Standard practice is the reverse: fix a large, well-understood e (e.g., 65537) and compute $d = e^{-1} \bmod \varphi(n)$, which is essentially always large. If CRT-RSA speed is the actual goal, achieve it via the CRT decomposition of Lesson 1 (Section 1.2.3) instead — that optimization does not shrink d itself, only the size of the exponents used *internally* during decryption.

2.2 Håstad's Broadcast Attack

2.2.1 Setup

Suppose a sender uses a **small public exponent** (e.g., $e = 3$) and sends the *same* message m, encrypted under e, to e or more different recipients with pairwise coprime moduli $n_1, \ldots, n_e$ (their own independent RSA keys):

$$c_i = m^e \bmod n_i, \qquad i = 1, \ldots, e.$$

Theorem 2.2.1 (Håstad's broadcast attack, $e = 3$ identical-message case)**.** *If $m < \min(n_1, n_2, n_3)$, an eavesdropper who observes c_1, c_2, c_3 (and the public moduli) recovers m in polynomial time, without knowing any private key.*

Proof idea. By CRT (Lesson 1, Theorem 1.1.7), the system $x \equiv c_i \pmod{n_i}$ for $i = 1, 2, 3$ has a unique solution C modulo $N = n_1 n_2 n_3$, computable directly from the public c_i, n_i values. Since $m^3 \equiv c_i \equiv C \pmod{n_i}$ for all i, and $m^3 < N$ (because $m < \min n_i \leq \sqrt[3]{N}$ roughly, precisely because $m^3 < n_1 n_2 n_3 = N$), we conclude $C = m^3$ *exactly as an integer*, not merely modulo N. An ordinary integer cube root recovers m. □

Example 2.2.2 (Håstad's attack, $e = 3$, fully worked)**.** *Let three (toy) recipients have moduli $n_1 = 143$, $n_2 = 323$, $n_3 = 667$ (pairwise coprime by construction: $143 = 11 \cdot 13$, $323 = 17 \cdot 19$, $667 = 23 \cdot 29$ — no shared prime factors). A sender broadcasts $m = 17$ to all three under $e = 3$:*

$$c_1 = 17^3 \bmod 143 = 51, \qquad c_2 = 17^3 \bmod 323 = 68, \qquad c_3 = 17^3 \bmod 667 = 244.$$

An eavesdropper computes $N = n_1 n_2 n_3 = 30{,}808{,}063$ and solves the CRT system to get $C = 4913$. Since $m^3 = 17^3 = 4913 < N$, this C is the true integer value of m^3 (no wraparound occurred). Taking the integer cube root: $\sqrt[3]{4913} = 17 = m$. ***No private key was ever touched.***

Remark 2.2.3 (Generalizing beyond identical messages)**.** *The identical-message case above is the simplest illustration. If instead each recipient gets a* linearly related *message (e.g., $m_i = a_i m + b_i$ for known public a_i, b_i — a common real-world pattern, since padding schemes or per-recipient headers often relate messages this way), the same underlying idea extends via a more general CRT-plus-resultant construction. This generalization is due to Håstad, and its* unknown-relation *extension (where even the relationship between messages is not fully known, only that it is small) is precisely Coppersmith's short-pad attack, previewed at the end of this lecture and building on the lattice technique in Section 2.6.*

Practical consequence. Never use a small public exponent to send the same (or simply related) message to multiple recipients without randomized padding. OAEP (Lesson 3) defeats this attack by making every ciphertext of the same message under the same key look independently random, so no algebraic relationship exists across ciphertexts for CRT to exploit.

2.3 Common Modulus Attack

2.3.1 Setup

A tempting (and wrong) efficiency idea: have a central authority generate a single modulus n, and give each user their own (e_i, d_i) pair on the *same* n, to save on key-generation cost. Suppose a message m is (perhaps accidentally, perhaps because the sender does not know which key to use) encrypted under two different users' exponents e_1, e_2 on the shared n, with $\gcd(e_1, e_2) = 1$:

$$c_1 = m^{e_1} \bmod n, \qquad c_2 = m^{e_2} \bmod n.$$

Theorem 2.3.1 (Common modulus attack)**.** *If* $\gcd(e_1, e_2) = 1$*, an eavesdropper who knows only* (n, e_1, e_2, c_1, c_2) *— no private key at all — recovers* m*.*

Proof idea. By the Extended Euclidean Algorithm, compute integers a, b (one necessarily negative) with $ae_1 + be_2 = 1$. Then

$$c_1^a c_2^b \equiv m^{ae_1} m^{be_2} \equiv m^{ae_1+be_2} \equiv m^1 \equiv m \pmod{n}.$$

A negative exponent, say $b < 0$, is handled by computing $c_2^{-1} \bmod n$ first (via the Extended Euclidean Algorithm again — this requires $\gcd(c_2, n) = 1$, true with overwhelming probability for a random message) and then raising that inverse to the power $|b|$. □

Example 2.3.2 (Common modulus attack, fully worked)**.** *Let* $n = 3233$ *(our familiar* $p = 61, q = 53$ *toy modulus). Two users share* n *with* $e_1 = 17$*,* $e_2 = 7$ *(coprime). A message* $m = 42$ *is encrypted under both:* $c_1 = 42^{17} \bmod 3233 = 2557$*,* $c_2 = 42^7 \bmod 3233 = 240$*.*

Extended Euclid on $(17, 7)$ *gives* $a = -2, b = 5$ *(check:* $-2 \cdot 17 + 5 \cdot 7 = -34 + 35 = 1$*). Since* $a < 0$*, compute* $c_1^{-1} \bmod n = 2160$ *(via modular inverse), then*

$$m = (c_1^{-1})^2 \cdot c_2^5 \bmod n = 42.$$

Recovered exactly — ***using no private key whatsoever****, just the two public exponents and two ciphertexts.*

Practical consequence. Never share a modulus n across users or purposes, even with different exponents. This design pattern occasionally resurfaces in embedded/legacy systems trying to save key-generation cost, but it is fundamentally broken the moment any two exponents used on the shared modulus are coprime — which is the common case, since exponents are typically chosen small and prime.

2.4 Cycling Attack (Conceptual)

2.4.1 Setup

This attack requires no assumption about d, e, or shared moduli — only encryption itself, and is presented mainly for conceptual completeness (it is essentially never practical at real key sizes, but it is instructive about the group-theoretic structure of RSA).

Theorem 2.4.1 (Cycling attack)**.** *Given only (n, e, c) where $c = m^e \bmod n$, repeatedly compute $c_1 = c^e \bmod n$, $c_2 = c_1^e \bmod n$, and so on, until some $c_k = c$. Then $m = c_{k-1}$.*

Proof idea. Encrypting repeatedly under the public exponent is the same operation as encrypting-then-encrypting: writing $c_0 = c$, we have $c_i \equiv m^{e^{i+1}} \pmod n$. Let k be the smallest positive integer with $c_k \equiv c_0 \pmod n$, i.e. $m^{e^{k+1}} \equiv m^e \pmod n$. Assuming $\gcd(m, n) = 1$, this means $e^{k+1} \equiv e \pmod{\operatorname{ord}_n(m)}$. Since e was originally chosen coprime to $\varphi(n)$ (and hence to $\operatorname{ord}_n(m)$, which divides $\lambda(n) \mid \varphi(n)$), e is invertible modulo $\operatorname{ord}_n(m)$, so we may cancel one factor of e from both sides: $e^k \equiv 1 \pmod{\operatorname{ord}_n(m)}$. Therefore

$$c_{k-1} \equiv m^{e^k} \equiv m^1 \equiv m \pmod n,$$

so the value immediately *before* the sequence closes the cycle is exactly the original plaintext. □

Example 2.4.2 (Cycling attack, fully worked)**.** *Let $p = 47, q = 59$, so $n = 2773$, $\varphi(n) = 2668$, and let $e = 17$. A message $m = 123$ is encrypted: $c = 123^{17} \bmod 2773 = 1924$.*

*Repeatedly raising to the e-th power: $c_1 = c^{17} \bmod n, c_2 = c_1^{17} \bmod n, \ldots$ — after **44** iterations, the sequence returns to $c_{44} = 1924 = c$. The value one step before, $c_{43} = 123 = m$, is the recovered plaintext.*

Why this stays purely conceptual. The cycle length here is the multiplicative order of e in $(\mathbb{Z}/\lambda(n)\mathbb{Z})^*$, which for well-chosen keys (large random primes, standard e) is astronomically large — comparable to $\varphi(n)$ itself in the worst case for the attacker. At real key sizes this attack is utterly infeasible; it survives in courses because it cleanly demonstrates that RSA encryption is literally iteration inside a finite cyclic structure, and it foreshadows why key-generation standards historically recommended checking that $\lambda(n)$ has large prime factors (to keep every element's order large) — the same underlying concern as smooth $p - 1$ in Pollard's method (Lesson 1).

2.5 The Franklin–Reiter Related-Message Attack

2.5.1 Setup

Suppose the *same recipient* (same n, e) receives two ciphertexts of two messages known to satisfy a **publicly known linear relationship** $m_2 = m_1 + t$ for some known constant t (e.g., two near-identical form-letter messages differing only in a numeric field), both under a small exponent, e.g. $e = 3$:

$$c_1 = m_1^3 \bmod n, \qquad c_2 = (m_1 + t)^3 \bmod n.$$

Theorem 2.5.1 (Franklin–Reiter, 1996)**.** *Given $(n, e{=}3, t, c_1, c_2)$, m_1 can be recovered in polynomial time, without factoring n.*

Proof idea. Define two polynomials over $\mathbb{Z}_n[x]$:

$$f_1(x) = x^3 - c_1, \qquad f_2(x) = (x + t)^3 - c_2.$$

Both have $x = m_1$ as a root (by construction), so $(x - m_1)$ divides $\gcd(f_1, f_2)$ in $\mathbb{Z}_n[x]$. Generically (i.e., unless the two polynomials share some *other* accidental common root, which happens with negligible probability for random instances), $(x - m_1)$ *is* the entire gcd, up to a constant multiple — a linear polynomial whose root is read off directly. The gcd is computed via the polynomial Euclidean algorithm, exactly analogous to the integer Euclidean algorithm used in Wiener's and the common-modulus attack, just one level of abstraction up (coefficients instead of integers, but the same division-with-remainder machinery). □

Example 2.5.2 (Franklin–Reiter, fully worked)**.** *Let $n = 3233$, $e = 3$, and suppose $m_1 = 20$, with a second message $m_2 = m_1 + t = 25$ for known offset $t = 5$. Ciphertexts: $c_1 = 20^3 \bmod 3233 = 1534$, $c_2 = 25^3 \bmod 3233 = 2693$.*

*Build $f_1(x) = x^3 - 1534$ and $f_2(x) = (x + 5)^3 - 2693 = x^3 + 15x^2 + 75x + (125 - 2693)$ over $\mathbb{Z}_{3233}[x]$. Running the polynomial Euclidean algorithm (dividing f_1 by f_2, then continuing with remainders, all coefficient arithmetic done mod 3233) reduces the pair to a single **linear** polynomial: $525x + 2432 \equiv 0 \pmod{3233}$. Solving: $x \equiv -2432 \cdot 525^{-1} \equiv 20 \pmod{3233}$ — exactly m_1. **Neither n was factored nor any private key touched** — only public ciphertexts, the known offset t, and polynomial arithmetic.*

Practical consequence. Never encrypt structurally related plaintexts (form letters, sequential IDs, incrementing counters) under a small exponent without randomized padding. This is one of the strongest practical arguments for OAEP: it destroys any algebraic relationship between the padded plaintexts, even when the underlying messages are related, which is exactly what this attack (and its harder unknown-relation cousin below) depends on.

2.6 Coppersmith's Method: An Overview

2.6.1 The General Problem

Coppersmith's small-root theorem (informal statement). Given a polynomial $f(x)$ of degree δ modulo n (with n of unknown or partially unknown factorization), if f has a root x_0 with $|x_0| < n^{1/\delta}$, then x_0 can be found in time polynomial in $\log n$ and δ.

This single theorem — due to Coppersmith (1996), with a simplified proof by Howgrave-Graham — underlies an entire family of RSA attacks that all reduce to "find a small root of a polynomial modulo n":

- **Håstad's short-pad attack** (the unknown-relation generalization of Håstad's broadcast attack above): recovers m when two related messages are sent under low e even when the *relationship itself* is not fully known, only that it is small.
- **Partial key exposure**: given enough of the high-order (or low-order) bits of p, recover the rest of p and factor n — directly relevant to side-channel and cold-boot attacks where an attacker recovers a fraction of the key bits.
- **Boneh–Durfee**: an improvement on Wiener's bound, extending the recoverable range of small d from $n^{0.25}$ up to $n^{0.292}$, using a two-variable version of Coppersmith's technique.
- **Stereotyped-message attacks**: recovering m when only a small unknown portion of a structured plaintext (e.g., a fixed-format message with one unknown short field) is unknown.

2.6.2 The Reduction to Lattices

From polynomial roots to short vectors. The key trick: build several polynomials $g_1(x), g_2(x), \ldots, g_k(x)$, all sharing the same root x_0 mod n (typically shifts and powers of the original f, multiplied by powers of n to control the modulus), and treat their *coefficient vectors* as a basis for a lattice $L \subset \mathbb{Z}^k$. A theorem of Howgrave-Graham says: if $h(x) = \sum h_i x^i$ is an integer linear combination of the g_i (hence still has x_0 as a root mod n) with **small enough Euclidean norm** $\|h\| < n/\sqrt{k}$, then $h(x_0) = 0$ holds as an equation over $\mathbb{Z}$, *not just modulo* n. Since h is now known to vanish at x_0 as an ordinary integer polynomial, standard root-finding recovers x_0 directly.

So the whole problem reduces to: **find a short vector in the lattice** L spanned by the g_i's coefficient vectors. This is precisely Shortest Vector Problem territory — and in practice, the LLL algorithm (an approximate-SVP solver) is exactly the tool used, since Coppersmith's bound is calibrated to what LLL can reliably deliver in polynomial time. If you have covered lattice-based cryptography, this is the same LLL you met there, now redeployed as an attack tool rather than a hardness-assumption underpinning.

Example 2.6.1 (Toy illustration of the core mechanism — small-scale, not full LLL)**.** *Suppose we know p shares its top bits with a known guess $\tilde{p}$: specifically $p = \tilde{p} + x_0$ for some small unknown x_0, and we are told $|x_0| < n^{1/2}$ (deliberately generous, purely for illustration). Consider $f(x) = \tilde{p} + x$, whose root mod p is x_0 by construction. In the real attack, one builds a small lattice from shifted copies of f, powers of n or p, and known bounds on x_0, then runs LLL to extract a short combination $h(x)$ satisfying Howgrave-Graham's norm bound; solving $h(x_0) = 0$ over $\mathbb{Z}$ (an ordinary, easy root-finding step once h is small) recovers x_0, hence p, hence the full factorization of n.*

At the tiny numeric scale used throughout this lecture (3–4 digit moduli), the lattices involved collapse to dimension 1 or 2 and the bound on x_0 needed for a compelling illustration is

not usefully separable from simple brute-force search — so a genuinely small worked numeric example does not illustrate the mechanism honestly. This is a case where ***the technique's value only appears at realistic scale*** *(hundreds of bits), which is exactly why Coppersmith's method is taught as a reduction to a problem you already have a tool for (LLL), rather than by hand-computation as in the earlier attacks in this lecture.*

Remark 2.6.2 (Why this belongs at the end of this lecture, not the start)**.** *Every attack earlier in this lecture (Wiener, Håstad, common modulus, Franklin–Reiter) can be fully hand-verified with small numbers, which is why they came first. Coppersmith's method is the most powerful of the family but the least hand-computable — its correctness rests on LLL actually finding a short-enough vector, which is a statement about lattice reduction quality, not elementary number theory. This is intentionally the conceptual capstone of RSA's mathematical attacks, and it is the point where number theory hands off to lattice reduction (the LLL algorithm) as the working tool.*

2.7 Summary and Bridge to Lesson 3

Summary

- **Wiener's attack**: a small d is recoverable directly from (n, e) via continued fractions — never choose d for efficiency.
- **Håstad's broadcast attack**: low exponent + repeated/related messages to multiple recipients + no padding = full break via CRT.
- **Common modulus attack**: sharing n across users with coprime exponents is a full break via the Extended Euclidean Algorithm.
- **Cycling attack**: a conceptual reminder that RSA encryption is iteration in a finite group — impractical, but foundational.
- **Franklin–Reiter**: known-relation messages under low exponent break via polynomial GCD.
- **Coppersmith's method**: the general small-root machinery underlying stronger versions of several of the above, reducing to LLL lattice reduction.

The unifying lesson: every attack here targets a specific *misuse* of RSA — small exponents without padding, small private exponents, repeated moduli, related plaintexts. None of them factor n, and all of them are defeated by proper randomized padding (OAEP) and standard key-generation discipline (large random d, non-shared n, large e such as 65537).

Looking Ahead — Lesson 3 — Implementation and Protocol Attacks on RSA

Next lecture assumes RSA is used *correctly* at the mathematical level (proper key generation, no shared moduli, no related-message broadcasting) and instead breaks it through its physical or protocol-level implementation:

- **Bleichenbacher's padding oracle attack** on PKCS#1 v1.5 — recovering plaintext by observing only whether a server accepts or rejects a padding format.
- **Timing attacks** exploiting the data-dependent branching in square-and-multiply (flagged back in Lesson 1).
- **Fault attacks** on CRT-RSA (the Bellcore attack) — a single computational glitch during the m_p or m_q branch (Lesson 1, Section 1.2.3) that factors n from one faulty signature.
- **ROCA** and other real-world key-generation vulnerabilities.
- OAEP's security argument, and why it defeats every low-exponent attack in today's lecture.

Tutorial Exercises

1. Run Wiener's attack by hand on $n = 100822548703$ with $e = 74531608157$ — hint: the continued fraction expansion has fewer than 10 terms. (Verify your recovered d by checking $ed \equiv 1$ modulo your recovered $\varphi(n)$.)
2. In the Håstad broadcast example, what is the minimum number of recipients needed if $e = 5$ instead of 3? Explain why in terms of the CRT bound $m^e < N$.

3. Extend the common modulus attack: what happens if $\gcd(e_1, e_2) = g > 1$? Is the attack still possible, and if so what can be recovered?

4. In the cycling attack, express the cycle length k precisely in terms of the multiplicative order of e modulo $\lambda(n)$ (the Carmichael function), and explain why choosing p, q such that $\lambda(n)$ has a large prime factor defends against short cycles.

5. Redo the Franklin–Reiter worked example with $e = 3$, $t = 7$, $m_1 = 30$ on the same $n = 3233$. Verify your recovered m_1.

6. Research question: read the original Boneh–Durfee paper's abstract and result statement. What is the improved bound on d compared to Wiener's $n^{0.25}$, and what extra mathematical tool (beyond one-dimensional continued fractions) does it require?

Lesson 3: The RSA Cryptosystem III — Implementation and Protocol Attacks

Padding Oracles, Timing and Fault Attacks, ROCA, OAEP, and the Guillou–Quisquater ZKP Bridge

Roadmap

(1) Padding oracles: PKCS#1 v1.5 and Bleichenbacher's "million-message" attack

(2) Timing attacks: leaking secret exponent bits through execution time, and blinding as the fix

(3) Fault attacks: the Bellcore attack on CRT-RSA signatures

(4) ROCA: a real-world key-generation flaw that turned Coppersmith's method (Lesson 2) into a practical factoring attack

(5) Side-channel countermeasures: a consolidated checklist

(6) OAEP: a security sketch for the padding scheme that defeats almost everything in Lessons 2–3

(7) Guillou–Quisquater: an RSA-based zero-knowledge proof of identity, and why it needs only one exponentiation-sized challenge space instead of many rounds

What every attack in this lecture has in common: n is properly generated, d is large, no modulus is shared, no message is broadcast — every mathematical precondition from Lessons 1–2 is satisfied. These attacks instead exploit *how RSA is computed* (timing, faults) or *how a protocol reacts to malformed input* (padding oracles), or a subtle flaw in *how the keys themselves were generated* (ROCA). This is the layer of RSA security that pure number theory cannot see.

3.1 Padding Oracles: Bleichenbacher's Attack on PKCS#1 v1.5

3.1.1 PKCS#1 v1.5 Encryption Padding

Recall from Lesson 1 that textbook RSA is deterministic and malleable, and that real systems pad the message before encrypting. The older padding standard, **PKCS#1 v1.5**, formats a k-byte block (where k is the byte-length of n) as

$$\texttt{EB} = \texttt{0x00} \,\|\, \texttt{0x02} \,\|\, \text{PS} \,\|\, \texttt{0x00} \,\|\, D,$$

where D is the message, PS is a string of *non-zero* random padding bytes at least 8 bytes long (padding out the block to exactly k bytes), and the leading `0x00 0x02` is a fixed, publicly-known format marker. The recipient decrypts c, obtains $\text{EB} = c^d \bmod n$ (represented as a k-byte string), and checks the format: does it start with `0x00 0x02`, is there a `0x00` byte terminating the padding within a valid range, etc. If the format is invalid, many real implementations (TLS servers being the classic case) return a distinguishable error — *this is the oracle.*

Definition 3.1.1 (Padding oracle)**.** *A **padding oracle** for PKCS#1 v1.5 is any mechanism (an error message, a timing difference, a connection reset — the specific channel does not matter) that lets an attacker submit a chosen ciphertext c and learn a single bit:* does $c^d \bmod n$ decrypt to a validly-PKCS#1-padded block, i.e. does it start with `0x00 0x02`?

Example 3.1.2 (A tiny PKCS#1 v1.5 block, made concrete)**.** *Take a toy $k = 8$-byte block (a real RSA-1024 block would use $k = 128$; 8 bytes is enough to see the structure). Encoding the 2-byte message $D =$ `0x1234` with 4 bytes of random non-zero padding PS = `0xA1 0x7F 0x3C 0x99` gives*

$$EB = \underbrace{\texttt{0x00}}_{1\text{ byte}}\,\underbrace{\texttt{0x02}}_{1\text{ byte}}\,\underbrace{\texttt{0xA1 0x7F 0x3C 0x99}}_{PS,\ 4\text{ bytes}}\,\underbrace{\texttt{0x00}}_{1\text{ byte}}\,\underbrace{\texttt{0x1234}}_{\text{message, 2 bytes}},$$

exactly 8 bytes. A recipient decrypting some ciphertext c' and obtaining, say, `0x00 0x02 0x00 ...` (a zero byte immediately after the `0x02` marker, where PS must be non-zero) rejects it as invalid — this is precisely the format check the padding oracle exposes as a single accept/reject bit, with nothing about the message D itself revealed directly by the check.

Why one bit is catastrophic. A one-bit oracle sounds harmless. Bleichenbacher (CRYPTO 1998) showed it is not: an attacker who can query the oracle on ciphertexts of the form $c' = c \cdot s^e \bmod n$ for chosen *blinding* values s learns, for each query, whether $m \cdot s \bmod n$ lands in the narrow range $[2B, 3B)$ where $B = 2^{8(k-2)}$ (i.e., whether the padded value "looks like" `0x00 0x02...`). Each YES answer is a strong constraint on the unknown m. Repeating this adaptively — using each new constraint to choose better values of s — narrows the set of possible m until only one remains. This is a classic **adaptive chosen-ciphertext attack**: it uses the decryption oracle itself, not any weakness in the RSA problem.

3.1.2 The Attack, at a High Level

Remark 3.1.3 (Why "million-message attack")**.** *Bleichenbacher's original paper estimated roughly 2^{20} (about a million) oracle queries to fully decrypt one ciphertext against 1024-bit RSA-PKCS#1v1.5 — expensive but entirely practical for an attacker with network access to a server that leaks the oracle bit, e.g. via a distinguishable TLS alert message ("bad padding" vs. other decryption failures). Later refinements (Bardou et al., 2012; ROBOT, 2017) reduced the required query count and revealed the attack was still exploitable in production TLS stacks nearly 20 years after publication — the flaw is in the* error-handling contract *of the protocol, which is easy to get subtly wrong even when the padding format itself is implemented correctly.*

Algorithm 7 Bleichenbacher's Attack (structure, not full case analysis)

1: **Input:** target ciphertext $c = m^e \bmod n$; oracle $\text{Valid}(c') \in \{0, 1\}$ testing PKCS#1 conformance
2: **Step 1 (blinding search):** find the smallest s_1 such that $c \cdot s_1^e \bmod n$ passes the oracle
3: **Step 2 (interval narrowing):** maintain a shrinking set of intervals that must contain m; each oracle query on a candidate s_i either confirms an interval or splits it further, using the known structure $[2B, 3B)$
4: **Step 3 (interval refinement):** once a single interval remains, search increasingly precise values of s to shrink the interval toward a single point
5: **Return** the unique m remaining once the interval has width 1

Practical consequence. Never let decryption failure reasons be distinguishable to the attacker — not via error message content, not via response timing, not via connection behavior. The historically recommended mitigation for legacy PKCS#1 v1.5 is to proceed *as if* decryption succeeded on failure (substituting random plaintext) so that no observable signal exists, rather than trying to enumerate and suppress every possible error channel. The complete fix, however, is to stop using PKCS#1 v1.5 for new designs and use OAEP (Section 3.6), whose security proof directly rules out this class of attack.

3.2 Timing Attacks

3.2.1 The Leakage Source

Recall the square-and-multiply algorithm from Lesson 1: the `if` branch executes an extra multiplication exactly when the current secret exponent bit is 1. Even without a visible branch, the multiplication and squaring operations themselves can take data-dependent time (due to conditional reduction steps, cache access patterns, or hardware multiplier behavior on operands of different sizes/structure).

Kocher's Timing Attack (1996), core idea. An attacker who can submit many ciphertexts $c_1, c_2, \ldots$ to a server and measure the wall-clock time T_i each decryption/signature takes can recover the secret exponent d **one bit at a time**, from the most significant bit downward. Having correctly guessed the top j bits of d, the attacker can *simulate* the modular exponentiation up to that point for each c_i and correlate the simulated intermediate computation's cost with the real measured T_i. A correct bit guess produces a statistically detectable correlation (the simulated step matches what the real hardware actually did); a wrong guess does not. Repeating bit by bit, with enough samples per bit to beat measurement noise, recovers all of d.

Remark 3.2.1 (Why this generalizes well beyond the toy square-and-multiply)**.** *The same statistical idea applies to* any *implementation where some intermediate computational step's cost depends on secret bits — including CRT-RSA's d_p, d_q exponentiations (arguably a richer target, since each exponent is only half the length, so fewer bits need to be recovered per correlated measurement), and even to cache-timing variants that measure which memory lines were touched rather than wall-clock time directly (e.g., a colocated attacker on the same physical machine observing cache eviction patterns).*

Example 3.2.2 (Toy illustration of the bit-by-bit correlation)**.** *Let the secret exponent be the 6-bit toy value $d = 101101_2 = 45$, used with right-to-left square-and-multiply (Lesson 1, Algorithm 2). Processing d's bits from the* least*-significant end, the algorithm performs a squaring every step, plus an* extra *multiplication exactly on the steps where the bit is* 1*: reading $d = 101101_2$ least-significant-bit first gives the bit sequence $1, 0, 1, 1, 0, 1$, so extra multiplications occur on steps $1, 3, 4, 6$ and are skipped on steps $2, 5$. An attacker who has correctly guessed the first few (low-order) bits can simulate exactly which steps* should *include the extra multiplication; timing many real executions and checking whether the measured cost is high (extra multiply present) or low (skipped) on the predicted step is precisely the one-bit correlation test Kocher's attack performs, repeated bit by bit until all 6 bits of d are recovered. At real key sizes (d having thousands of bits, not 6), each bit requires many timing samples to beat measurement noise, but the mechanism is identical to this toy trace.*

3.2.2 Defense: Blinding

Definition 3.2.3 (RSA blinding)**.** *Before computing $m = c^d \bmod n$, the signer/decryptor picks a fresh random r (coprime to n) for every operation, computes*

$$c' = c \cdot r^e \bmod n, \qquad m' = (c')^d \bmod n, \qquad m = m' \cdot r^{-1} \bmod n.$$

Proposition 3.2.4 (Correctness of blinding)**.** $m' = (c \cdot r^e)^d = c^d \cdot r^{ed} \equiv c^d \cdot r \pmod{n}$ *(using $r^{ed} \equiv r \pmod{n}$ by the same Euler's-theorem identity from Lesson 1 that makes RSA correct in the first place), so $m' \cdot r^{-1} \equiv c^d \equiv m \pmod{n}$.*

Why blinding defeats the timing attack. The exponentiation the attacker times is now $(c')^d \bmod n$ for a value c' that the attacker cannot predict or control (it is masked by a fresh random r chosen internally, unknown to the attacker, on every single call). Since the attacker

cannot correlate the timing of a specific known input against a specific bit-guess simulation — the actual base being exponentiated is different and unknown every time — the statistical correlation Kocher's attack relies on collapses to noise. **Blinding does not make the exponentiation itself constant-time**; it removes the attacker's ability to choose or know the input being timed, which is what the attack actually needs.

Example 3.2.5 (Blinding, fully worked)**.** *Reuse the familiar toy key* $p = 61, q = 53, n = 3233, e = 17, d = 2753$*. An attacker sends ciphertext* $c = 65^{17} \bmod 3233 = 2790$ *(encryption of* $m = 65$*) and wants to time the server's decryption of exactly this* c*.*

The server instead picks a fresh random blinding factor $r = 41$ *(coprime to* n*) and computes*

$$c' = c \cdot r^{17} \bmod 3233 = 2130.$$

It is $c' = 2130$ *— not the attacker's original* $c = 2790$ *— whose decryption the server actually times:* $m' = (c')^d \bmod n = 2130^{2753} \bmod 3233 = 2665$*. Unblinding recovers the true answer:*

$$m = m' \cdot r^{-1} \bmod n = 2665 \cdot 41^{-1} \bmod 3233 = 65,$$

correctly matching the original message. ***The attacker's timing measurement is on*** $c' = 2130$***, a value they never chose and cannot predict*** *— a fresh random* r *is drawn on every call, so the same* c *is never timed twice under the same disguise, and no bit-guess simulation the attacker builds around their chosen* c *can be correlated against what the server actually computed.*

Practical consequence. Production RSA implementations combine *two* defenses, not one: (1) blinding, to remove attacker control over the timed input, and (2) constant-time modular exponentiation (e.g., Montgomery ladder instead of naive square-and-multiply, avoiding secret-dependent branches or table lookups entirely) as defense in depth, since blinding alone does not stop attackers who can observe *which physical operations occurred* rather than just aggregate time (e.g., some cache/microarchitectural attacks). Real-world CVEs against RSA timing side channels (e.g., against OpenSSL's Montgomery multiplication in various years) have repeatedly targeted implementations that had blinding but incomplete constant-time arithmetic underneath it.

3.3 Fault Attacks: The Bellcore Attack on CRT-RSA

3.3.1 Setup

Recall CRT-RSA decryption/signing from Lesson 1 (Section 1.2.3): the signer computes

$$s_p = c^{d_p} \bmod p, \qquad s_q = c^{d_q} \bmod q,$$

and recombines via CRT to get the full signature s. This is roughly $4\times$ faster than computing $c^d \bmod n$ directly — which is precisely why real implementations use it, and precisely what makes the following attack possible.

Theorem 3.3.1 (Bellcore attack, Boneh–DeMillo–Lipton, 1997)**.** *Suppose an attacker can induce a single transient computational fault (via voltage glitching, clock glitching, laser fault injection, or simply naturally-occurring hardware error) during* exactly one *of the two CRT branches — say the s_p computation — while the s_q branch computes correctly, and can obtain the resulting faulty signature s' on a message m whose value the attacker knows. Then the attacker recovers the prime factor q (and hence the full private key) in polynomial time, using only s', m, and the public (n, e) — **no correct signature is needed at all.***

Proof idea. Since only the s_p branch was corrupted, the faulty s' still satisfies $s' \equiv m^d \pmod q$ (the q-branch was untouched), but $s' \not\equiv m^d \pmod p$ (the p-branch was corrupted, and a random fault essentially never happens to still satisfy the same congruence). Hence

$$q \mid (s'^e - m), \qquad p \nmid (s'^e - m)$$

(the second holds because $s'^e \equiv m \pmod q$ requires $s' \equiv m^d \pmod q$ by RSA correctness restricted mod q, which holds, but the same argument mod p fails since s' is wrong there). Therefore $\gcd(s'^e - m \bmod n,\ n) = q$ exactly — the Euclidean algorithm reveals the fault-affected prime directly, and from q the attacker computes $p = n/q$, then the entire private key. □

Example 3.3.2 (Bellcore attack, fully worked)**.** *Using the familiar toy key $p = 61, q = 53, n = 3233, e = 17, d = 2753$, with $d_p = 53, d_q = 49, q^{-1} \bmod p = 38$. The signer correctly computes $s_p = 65^{53} \bmod 61 = 4$ and $s_q = 65^{49} \bmod 53 = 12$, recombining to the correct signature $s = 588$ (indeed $588^{17} \bmod 3233 = 65 = m$, as required).*

Now suppose a fault corrupts the p-branch only, flipping s_p to $s'_p = 5$ (one off from the correct value) while $s_q = 12$ is untouched. Recombining via the same CRT formula gives a faulty signature $s' = 2602$. Checking: $(s')^{17} \bmod 3233 = 1496$, and indeed $1496 \equiv 12 \equiv m \pmod{53}$ (the q-branch congruence survives) while $1496 \equiv 32 \not\equiv 4 \equiv m \pmod{61}$ (the p-branch congruence is broken).

The attacker, knowing only $n = 3233$, $e = 17$, the message $m = 65$, and the single faulty signature $s' = 2602$, computes

$$\gcd\left((s')^e \bmod n \ -\ m,\ n\right) = \gcd(1496 - 65,\ 3233) = \gcd(1431, 3233) = \mathbf{53} = q.$$

A single faulty signature factored the modulus completely *— recovering q, then $p = 3233/53 = 61$, then $\varphi(n)$ and d itself. No correct signature was ever needed.*

Practical consequence: signature verify-before-release. The standard defense is procedural, not algebraic: **always verify a freshly computed CRT-RSA signature against the public key before releasing it** — i.e. check $s^e \bmod n \stackrel{?}{=} m$ internally, and discard/retry silently if the check fails, releasing nothing. A genuine fault will (overwhelmingly likely) fail this check, so the faulty signature the attacker needs is simply never emitted. This single check closes the entire attack class at negligible extra computational cost (one public-exponent exponentiation, which is cheap since e is small, e.g. 65537), and is mandated by essentially every modern smart-card and HSM signing standard.

Remark 3.3.3 (Why this specifically targets CRT-RSA, and why that matters)**.** *Non-CRT RSA (computing* $c^d \bmod n$ *directly) is* also *vulnerable to fault attacks in principle, but recovering the key from a single faulty full-size exponentiation is far harder — there is no clean two-branch structure to exploit algebraically. This is precisely the trade-off flagged in Lesson 1: the* $4\times$ *CRT speedup is bought at the cost of a much sharper fault-attack surface, which is why smart cards and HSMs that use CRT-RSA* must *implement the verify-before-release countermeasure (or an equivalent redundant computation), while implementations that skip CRT entirely face a smaller (but nonzero) fault-attack risk.*

3.4 ROCA: When Key Generation Itself Is the Vulnerability

3.4.1 Background

ROCA ("Return of Coppersmith's Attack," Nemec et al., 2017) is not a flaw in RSA's mathematics or in a protocol's error handling — it is a flaw in a specific *key-generation implementation* (the Infineon RSALib used in many smart cards, TPMs, and government-issued ID cards) that made factoring *practically feasible* for affected keys, at sizes (512–2048 bits) that are otherwise completely safe against GNFS.

The vulnerable construction. The affected library generated primes of the special form

$$p = k \cdot M + \left(65537^a \bmod M\right)$$

for a secret integer k, a secret exponent a, and a fixed public constant M (a product of the first several hundred small primes — a "primorial"), apparently as a performance optimization for the prime-generation process on constrained smart-card hardware. This construction was undisclosed until reverse-engineered by researchers.

Proposition 3.4.1 (Why this collapses the search space)**.** *An attacker who knows the public modulus $n = pq$ and suspects it was generated this way does* not *need to search all $\approx \sqrt{n}$ possible values of p. Instead, since $p \bmod M$ is one of only $\mathrm{ord}_M(65537)$ possible residues (the multiplicative order of 65537 modulo M — typically a number in the thousands to low millions for the real ROCA parameters, versus M itself and $\sqrt{n}$ being astronomically larger), the attacker can:*

1. *Enumerate the (comparatively tiny) set of candidate residues $r_a = 65537^a \bmod M$.*
2. *For each candidate residue, use **Coppersmith's method** (Lesson 2, Section 2.6) to search for a small k completing $p = kM + r_a$ into an actual factor of n — Coppersmith's small-root machinery is exactly suited to this "most of p is unknown but constrained to a coset" structure.*

This turns a brute-force search over all of $\mathbb{Z}$ into a brute-force search over a small candidate set, composed with *a polynomial-time root-finding step per candidate — dropping the effective attack cost for affected 1024-bit keys from "infeasible" to a few CPU-hours, and for affected 2048-bit keys to a few CPU-months on modest hardware (the original paper's reported figures), rather than the millennia GNFS would require.*

Example 3.4.2 (Toy illustration of the fingerprint structure — not real ROCA scale)**.** *Let the (deliberately tiny, illustrative-only) constant be $M = 3{\cdot}5{\cdot}7{\cdot}11 = 1155$. Compute $65537 \bmod M = 857$; the multiplicative order of 857 modulo 1155 turns out to be 12 — meaning there are only **12** distinct residues $65537^a \bmod M$ ever reachable, out of the 1155 residues mod M in general (and utterly negligible next to $\sqrt{n}$ for a cryptographic-size n). Taking $a = 1$ (so the residue is $65537 \bmod 1155 = 857$) and $k = 2$ gives the candidate*

$$p = 2 \cdot 1155 + 857 = 3167,$$

which is in fact prime. An attacker facing a modulus $n = p \cdot q$ built this way would not need to guess p among all integers near $\sqrt{n}$ — only among the 12 residues mod 1155 (here, $\{1, 331, 362, 529, 593, 692, 694, 857, 923, 991, 1024, 1088\}$), each followed by a Coppersmith-style search for k. At real ROCA parameters (M a $\sim 2^{200}$-ish primorial, order in the low millions), the identical structural idea — not this exact toy arithmetic — is what makes the real attack practical.

Practical consequence. This vulnerability class does not indicate any weakness in RSA itself, in Coppersmith's method, or in any of the earlier lectures' key-generation advice (large random independent primes, avoiding smooth $p - 1$, avoiding shared/related moduli) — it is precisely what happens when an implementation *silently deviates* from that advice for an unstated performance reason. The lesson for practitioners: **never trust that "a widely deployed, certified library" generates primes uniformly at random** without independent verification; ROCA affected certified smart cards and government ID infrastructure for years before detection, precisely because the deviation was invisible from the public key alone unless someone specifically tested for the fingerprint.

3.5 Side-Channel Countermeasures: A Consolidated Checklist

The individual defenses mentioned throughout this lecture form a coherent set of standard practices. Collecting them here as a reference:

Threat	**Countermeasure**	**Covered in**
Padding-oracle CCA	OAEP instead of PKCS#1 v1.5; uniform error handling	Sec. 1, Sec. 6
Timing leakage	RSA blinding (randomize the input)	Sec. 2
Timing/cache leakage	Constant-time modular exponentiation (Montgomery ladder)	Sec. 2
CRT fault injection	Verify signature before release; redundant computation	Sec. 3
Weak key generation	Independent verification of prime-generation randomness; avoid undisclosed proprietary shortcuts	Sec. 4
General implementation drift	Use vetted, widely-audited cryptographic libraries; do not hand-roll modular exponentiation or padding logic	All sections

Remark 3.5.1 (The unifying theme, one more time)**.** *Every defense above targets the* gap between the mathematical specification of RSA and its physical realization*. Lessons 1–2 taught you when RSA is mathematically sound; this lecture teaches you that mathematical soundness is necessary but nowhere near sufficient — an implementation can be simultaneously "textbook-perfect" in its algebra and completely broken in its execution.*

3.6 OAEP: A Security Sketch

3.6.1 Construction

Optimal Asymmetric Encryption Padding (Bellare–Rogaway, 1994) replaces the fixed, structured PKCS#1 v1.5 padding with a randomized, "all-or-nothing"-looking transform, before the result is encrypted with textbook RSA:

OAEP encoding (simplified). To encode a message m (padded with a fixed-length string of zero bits PS for domain separation) using two hash functions G (a mask-generating function) and H, and a fresh random seed r:

$$X = (m \,\|\, \mathrm{PS}) \oplus G(r), \qquad Y = r \oplus H(X), \qquad \mathrm{EM} = X \,\|\, Y,$$

then EM (interpreted as an integer $< n$) is encrypted as ordinary textbook RSA: $c = \mathrm{EM}^e \bmod n$. Decryption inverts each XOR step in reverse (recovering r from Y and $H(X)$, then $m \,\|\, \mathrm{PS}$ from X and $G(r)$) and **rejects** if the recovered padding string does not exactly match the expected PS.

Example 3.6.1 (Toy illustration of the OAEP masking mechanism — not a real hash)**.** *The real G, H are cryptographic hash functions; to see the seed/mask mechanics with actual numbers, use toy 4-bit stand-ins $G(r) = (7r + 3) \bmod 16$ and $H(X) = (5X + 1) \bmod 16$ (fixed, public functions — not secret), omit PS for brevity, and encode the toy message $m = 5$ with fresh random seed $r = 9$:*

$$G(9) = 2, \qquad X = m \oplus G(r) = 5 \oplus 2 = 7, \qquad H(7) = 4, \qquad Y = r \oplus H(X) = 9 \oplus 4 = 13.$$

So $EM = (X, Y) = (7, 13)$. Decoding reverses the steps in the opposite order: $r' = Y \oplus H(X) = 13 \oplus 4 = 9$, then $m' = X \oplus G(r') = 7 \oplus 2 = 5$ — correctly recovering m.

Now encrypt the same *message $m = 5$ again with a fresh seed $r = 2$: $G(2) = 1$, so $X = 5 \oplus 1 = 4$; $H(4) = 5$, so $Y = 2 \oplus 5 = 7$; giving $EM = (4, 7)$ — completely different from $(7, 13)$ despite encoding the identical message.* ***This is exactly the property that defeats Håstad and Franklin–Reiter****: two encodings of the same (or related) plaintext no longer share any visible algebraic structure once OAEP has randomized them, before RSA is even applied.*

Security sketch, in the random oracle model. Modeling G and H as independent random oracles, Bellare and Rogaway showed OAEP-RSA achieves **IND-CCA2 security** (indistinguishability under adaptive chosen-ciphertext attack) whenever the underlying trapdoor permutation (here, RSA) is one-way. The proof idea: any adversary that can distinguish encryptions, or that submits a chosen ciphertext to a decryption oracle and gets useful information back, can be shown (via a simulation/extraction argument) to have queried the random oracle H or G on the specific value tied to the challenge ciphertext's underlying RSA pre-image — which lets a simulator extract an RSA inverse from the adversary's queries, contradicting the one-wayness of RSA. Informally: **any successful attack on OAEP-RSA can be converted into an algorithm that inverts plain RSA**, which we assume is hard.

Remark 3.6.2 (Why OAEP specifically defeats the earlier attacks)**.** *Each attack in Lessons 2–3 relied on some algebraic or structural regularity surviving into the encrypted value:*

- ***Håstad / Franklin–Reiter*** *(Lesson 2) needed related plaintexts to remain related after encoding. OAEP's random seed r makes two encryptions of related (or even identical) messages look like uniformly random, unrelated strings before RSA is even applied — there is no longer any algebraic relationship for CRT or polynomial-GCD tricks to exploit.*

- ***Bleichenbacher's padding oracle*** *(Section 1) exploited the fact that a large fraction of PKCS#1v1.5-decrypted garbage still "looks like" a differently-shaped valid message with nonzero probability, and that validity itself was cheaply, deterministically checkable and distinguishable. OAEP's redundancy check (the PS match) has negligible probability of accidentally validating on a random/incorrect decryption, and — critically — OAEP's* security proof *accounts for a decryption oracle directly (IND-CCA2 is defined with exactly such an oracle available to the adversary), which PKCS#1v1.5's original design did not.*
- ***Malleability and determinism*** *(flagged already in Lesson 1) are both eliminated: the random seed r makes encryption probabilistic, and the tightly-coupled XOR/hash structure means a modified ciphertext decrypts (if it decrypts at all) to something the redundancy check almost always rejects, rather than to a predictably related plaintext.*

What OAEP does *not* fix. OAEP is a defense at the *mathematical/protocol* layer. It does nothing by itself against Section 2's timing attacks or Section 3's fault attacks against the underlying modular exponentiation — those require blinding, constant-time arithmetic, and verify-before-release, independently of which padding scheme sits on top. Real-world security requires *all* of these layered together; OAEP alone is necessary but not sufficient.

3.7 Guillou–Quisquater: An RSA-Based Zero-Knowledge Proof

3.7.1 Motivation: Proving Identity Without Revealing the Secret

Everything so far in this RSA module has been about encryption or signing. Guillou–Quisquater (1988) uses the same algebraic structure — an RSA-type modulus and modular exponentiation — for a different goal: letting a prover convince a verifier *"I know a secret s tied to my claimed identity"* without revealing s, and without letting the verifier (or an eavesdropper) later impersonate the prover.

Setup. A trusted authority publishes an RSA-like modulus $n = pq$ and a public verification exponent v (coprime to $\varphi(n)$, exactly as with an RSA public exponent). Each prover's identity is mapped, via a public redundancy/hash function, to a value J; the authority (who alone knows $\varphi(n)$, i.e. p, q) issues the prover a secret credential s satisfying

$$s^v \cdot J \equiv 1 \pmod{n}, \qquad \text{i.e.} \qquad s \equiv J^{-1/v} \pmod{n},$$

computable by the authority (who can take v-th roots mod n using $\varphi(n)$) but not by anyone else, exactly as RSA decryption requires the private exponent.

Algorithm 8 Guillou–Quisquater Identification Protocol (one round)

1: **Prover** picks random r coprime to n, sends commitment $x = r^v \bmod n$
2: **Verifier** picks random challenge $c \in \{0, 1, \ldots, v-1\}$, sends c
3: **Prover** computes and sends $y = r \cdot s^c \bmod n$
4: **Verifier** accepts iff $x \equiv y^v \cdot J^c \pmod{n}$

Proposition 3.7.1 (Completeness)**.** *If the prover genuinely knows s and follows the protocol, $y^v \cdot J^c = (rs^c)^v \cdot J^c = r^v \cdot s^{vc} \cdot J^c = x \cdot (s^v J)^c \equiv x \cdot 1^c \equiv x \pmod{n}$, so an honest verifier always accepts.*

Proposition 3.7.2 (Soundness, sketch)**.** *A cheating prover who does not know s but wants to pass for a specific challenge value c can indeed fake a valid-looking transcript for* that one *value of c (by choosing y first, then setting $x = y^v J^c \bmod n$ to match) — but must commit to x* before *learning the verifier's actual random challenge. A cheating prover can therefore satisfy at most one of the v possible challenge values with a given commitment x, so guessing the verifier's challenge correctly happens with probability exactly $1/v$. Choosing v large (e.g. a value comparable in size to a standard RSA public exponent) drives the cheating probability negligibly low in a* single round*, unlike Fiat–Shamir-style protocols built on a $v = 2$ structure, which need many sequential rounds to reach the same soundness level.*

Remark 3.7.3 (Zero-knowledge, sketch)**.** *A simulator that does not know s can still produce transcripts (x, c, y) indistinguishable from real ones* without *interacting with a real prover: pick c and y uniformly at random first, then set $x = y^v J^c \bmod n$ (exactly the "rewind and fake the commitment" trick used in Proposition 2, but now run by the simulator rather than an attacker). Since x is computed from uniformly random y, c, the resulting triple has exactly the same distribution as a genuine protocol run, which is the essence of the honest-verifier zero-knowledge argument: nothing in the transcript could not have been produced without knowledge of s, so the transcript itself leaks no information about s.*

Example 3.7.4 (Guillou–Quisquater, fully worked)**.** *Reuse the familiar toy modulus $n = 3233$ ($p = 61, q = 53$, $\varphi(n) = 3120$), and take the public exponent $v = 7$ (check:* $\gcd(7, 3120) = 1$*, exactly the RSA-key-generation condition from Lesson 1). Suppose the authority has issued the prover secret $s = 100$ (with* $\gcd(100, 3233) = 1$*), corresponding to public identity value*

$$J \equiv s^{-7} \pmod{n} = 1818, \qquad \textit{check: } s^7 \cdot J \bmod n = 1.$$

Round: *the prover picks* $r = 37$ *and sends the commitment* $x = 37^7 \bmod 3233 = 1700$. *The verifier picks challenge* $c = 3$. *The prover, knowing* $s = 100$, *replies with* $y = 37 \cdot 100^3 \bmod 3233 = 1548$.

The verifier checks: $y^7 \cdot J^3 \bmod 3233 = 1548^7 \cdot 1818^3 \bmod 3233 = 1700 = x$. ***Match*** *— the verifier accepts, and at no point did* $s = 100$ *itself appear in any message exchanged.*

Example 3.7.5 (Why a cheating prover cannot do better than guessing)**.** *Suppose an impostor does* not *know* s*, but tries to cheat by anticipating the verifier's challenge. Working backwards, they pick an arbitrary response* $y = 222$ *and a guessed challenge* $c = 3$*, then compute the commitment that would make the check pass* for that guess*:*

$$x = y^7 \cdot J^3 \bmod 3233 = 937.$$

If the verifier happens to challenge with $c = 3$ *(matching the guess), the check passes:* $y^7 J^3 \bmod 3233 = 937 = x$. *But if the verifier instead challenges with, say,* $c = 5$*, the same fixed* $(x, y) = (937, 222)$ *fails:* $y^7 J^5 \bmod 3233 = 789 \neq 937$. *Since* x *was committed* before *the challenge was revealed, the impostor had to guess* c *in advance — correctly with probability exactly* $1/v = 1/7$*, matching Proposition 3.7.2's soundness claim precisely.*

Remark 3.7.6 (The bridge back to earlier material)**.** *Guillou–Quisquater is structurally the RSA-flavored cousin of the Fiat–Shamir identification scheme (which uses* $v = 2$ *and a modulus whose factorization is likewise the trapdoor) — the underlying commit/challenge/response skeleton (Algorithm 8 above) is the same three-move structure used by essentially every modern zero-knowledge identification and signature scheme, whatever the underlying hard problem (factoring/RSA here; discrete log for Schnorr/Fiat–Shamir; lattice problems for schemes built the same way from LWE/SIS). Recognizing this shared skeleton — commit, challenge, response, verify — is the single most transferable idea to carry out of this lecture.*

3.8 Summary

Summary

- **Padding oracles** (Bleichenbacher): a single bit of decryption-validity feedback, queried adaptively, fully decrypts PKCS#1v1.5 ciphertexts — the flaw is in error-handling, not in RSA's algebra.
- **Timing attacks** (Kocher): secret exponent bits leak through execution time of modular exponentiation; blinding (randomizing the input) plus constant-time arithmetic are both required as defense.
- **Fault attacks** (Bellcore): a single corrupted CRT branch, combined with one known message/signature pair, factors n outright via a single gcd computation; verify-before-release closes this completely.
- **ROCA**: a real-world case where non-uniform, undisclosed key generation collapsed the effective search space enough for Coppersmith's method (Lesson 2) to factor otherwise-safe key sizes.
- **OAEP**: randomized, redundancy-checked padding with an IND-CCA2 security proof (in the random oracle model) that directly rules out the algebraic and oracle-based attacks from this and the previous lecture.
- **Guillou–Quisquater**: the same RSA algebra, repurposed as a zero-knowledge identification protocol, with soundness error $1/v$ per round instead of $1/2$ — and structurally the same three-move skeleton as the lattice-based signatures earlier in the course.

The course-level takeaway. Across all three RSA lectures: correctness is a CRT argument (Lesson 1); security against *misuse* requires avoiding a specific list of structural traps (Lesson 2); and security in *deployment* requires an entirely separate discipline — constant-time arithmetic, blinding, fault detection, verified randomness in key generation, and provably secure padding (Lesson 3). None of these three layers substitutes for the others: a mathematically flawless, correctly-used RSA implementation can still be broken in milliseconds by a fault injector, and a side-channel-hardened implementation is still broken instantly if d is small or n is shared. Real-world RSA security is the conjunction of all three lectures, not any one of them alone.

Tutorial Exercises

1. In Bleichenbacher's attack, explain concretely why the oracle needs only to distinguish "starts with `0x00 0x02`" rather than fully validating the entire padding string, and why this makes the attack robust to minor implementation variations across different TLS stacks.
2. Suppose a timing attack has correctly recovered the top 6 bits of a 12-bit toy private exponent d used with square-and-multiply. Describe precisely what additional measurements the attacker needs to recover bit 7, and why the attack proceeds strictly most-significant-bit first.
3. Redo the Bellcore worked example with the fault instead corrupting the q-branch (s_q) rather than the p-branch, and show that the same gcd technique now recovers p instead of q.

4. Using $M = 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 = 15015$, compute the multiplicative order of 65537 mod M, and compare the resulting candidate-residue count against M itself and against $\sqrt{n}$ for a realistic 2048-bit RSA modulus. What does this comparison tell you about why ROCA is practical while generic factoring is not?

5. Explain why OAEP's random oracle security proof does not, by itself, guarantee security against the timing and fault attacks earlier in this lecture. What does this imply about the completeness of a security argument that only addresses the mathematical padding layer?

6. Compare Guillou–Quisquater's soundness argument to Fiat–Shamir's. If a Fiat–Shamir-style scheme uses $v = 2$ per round and needs t rounds to reach soundness error 2^{-t}, how many G-Q rounds are needed to reach the same soundness error using $v = 2^{16}+1 = 65537$? What communication-cost trade-off does this reveal?